



نام درس: شبکه های کامپیوتری
نویسنده: محمد امانلو - ۸۱۰۱۰۰۰۸۴



تمرین شماره ۳

۱ سؤال ۱ - نقش شماره پورت

پاسخ: لایه شبکه IP فقط یک کار اصلی انجام می دهد: رساندن بسته از یک میزبان به میزبان دیگر (Host-to-Host Delivery). این لایه با استفاده از آدرس IP، مقصد فیزیکی (کامپیوتر مقصد) را پیدا می کند و بسته را تا آن میزبان جلو می برد؛ اما اصلاً برایش مهم نیست که در آن میزبان، کدام برنامه یا کدام process باید این داده را دریافت کند. در عمل، روی هر کامپیوتر ممکن است همزمان چندین برنامه شبکه ای فعال باشند؛ مثل مرورگر وب، کلاینت ایمیل، یک برنامه SSH، یک chat app و ... اگر فقط آدرس IP را داشته باشیم، لایه شبکه بعد از تحویل بسته به میزبان مقصد نمی تواند تشخیص دهد که این بسته مثلاً مال مرورگر است یا مال کلاینت ایمیل؛ این «نیمه کاره بودن» سرویس در این جاست که خودش را نشان می دهد.

لایه انتقال (Transport Layer) با وارد کردن مفهوم «شماره پورت» (Port Number) این سرویس را از حالت «میزبان به میزبان» به «فرآیند به فرآیند» (Process-to-Process Delivery) گسترش می دهد. ایده به این صورت است:

□ هر برنامه یا سرویسی که می خواهد از شبکه استفاده کند، یک نقطه ی انتهایی به نام «سوکت» (Socket) ایجاد می کند. هر سوکت در یک میزبان با یک شماره پورت مشخص (Port Number) شناسایی می شود.

□ در سمت فرستنده، لایه انتقال داده ها را از چندین process مختلف دریافت می کند. این جمع کردن جریان های مختلف داده در یک کانال مشترک، «مالتی پلکسینگ» (Multiplexing) نام دارد. لایه انتقال برای هر قطعه ی داده، یک سگمنت (Segment) می سازد و در سرآیند آن (Header)، شماره پورت مبدأ و شماره پورت مقصد را قرار می دهد.

□ بعد، این سگمنت در داخل یک بسته ی IP قرار می گیرد و لایه شبکه آن را صرفاً بر اساس آدرس IP به میزبان مقصد می رساند. تا این جا هنوز فقط تحویل Host-to-Host انجام شده است.

□ در سمت گیرنده، بسته ی IP به لایه انتقال تحویل داده می شود. حالا نوبت «دی مالتی پلکسینگ» (Demultiplexing) است. لایه انتقال شماره پورت مقصد موجود در هدر سگمنت را نگاه می کند و با توجه به آن، سگمنت را به سوکت مناسب و در نهایت به process درست تحویل می دهد.

بنابراین، شماره پورت در واقع یک «شناسه منطقی» برای هر فرآیند شبکه‌ای روی یک میزبان است. لایه شبکه با آدرس IP فقط تضمین می‌کند که بسته به کامپیوتر درست برسد، اما لایه انتقال با استفاده از شماره پورت است که مشخص می‌کند این بسته باید به کدام برنامه برسد. به این ترتیب، سرویس لایه شبکه از تحویل Host-to-Host یک گام بالاتر می‌آید و در سطح دقیق‌تر تحویل Process-to-Process پیاده‌سازی می‌شود؛ یعنی داده نه فقط به «کامپیوتر درست»، بلکه به «فرآیند درست» در آن کامپیوتر تحویل داده می‌شود.

۲ سوال ۲ - نقش کنترل جریان و کنترل ازدحام در شبکه

پاسخ:

پروتکل TCP برای این که هم دید درون میزبانی را پوشش دهد و هم دید شبکه‌ای را، از دو مکانیسم کنترلی مجزا استفاده می‌کند: Flow Control (کنترل جریان) و Congestion Control (کنترل ازدحام). هر دو نهایتاً نرخ ارسال فرستنده را محدود می‌کنند، اما هدف و «کسی که از او محافظت می‌کنند» کاملاً متفاوت است.

۱) کنترل جریان (Flow Control)

هدف اصلی کنترل جریان این است که نگذارد فرستنده با سرعت ارسال بالای خودش، گیرنده را «خفه» کند. ممکن است گیرنده به هر دلیل (مثلاً سرعت کم CPU، برنامه کند، یا درگیر کارهای دیگر بودن) نتواند به همان سرعت داده‌ها را از بافر دریافت بخواند و پردازش کند. اگر در این وضعیت، فرستنده همچنان با نرخ بالا ارسال کند، بافر دریافت (Receive Buffer) در میزبان گیرنده پر می‌شود و ناچار بخشی از داده‌ها دور ریخته می‌شوند (سرریز بافر). TCP برای حل این مشکل از مکانیزمی به نام «پنجره دریافت» (Receive Window یا rwnd) استفاده می‌کند. ایده به این شکل است:

□ گیرنده در هر لحظه می‌داند چه قدر فضای خالی در بافر دریافت خود دارد.

□ این مقدار را در قالب پنجره دریافت (rwnd) در سگمنت‌های TCP به فرستنده اعلام می‌کند.

□ فرستنده موظف است حجم داده‌های «تأیید نشده» (Unacknowledged Data) که هم‌زمان در مسیر دارد، از این مقدار اعلام‌شده بزرگ‌تر نکند.

به این ترتیب، کنترل جریان مستقیماً از گیرنده محافظت می‌کند. یعنی اجازه نمی‌دهد بافر دریافت گیرنده بیش از ظرفیتش پر شود و داده‌ها به خاطر سرریز بافر از دست بروند.

۲) کنترل ازدحام (Congestion Control)

در کنترل ازدحام، زاویه نگاه عوض می شود. این جا مسئله دیگر فقط «یک گیرنده» نیست، بلکه «کل شبکه» و منابع مشترک آن (لینک ها، روترها و صف های داخلی آنها) مطرح است. ازدحام (Congestion) زمانی رخ می دهد که مجموع نرخ ترافیک ورودی به روترها از ظرفیت لینک های خروجی آنها بیشتر شود؛ در این حالت:

□ صف های روترها پر می شود،

□ تأخیر (Latency) زیاد و زیادتر می شود،

□ و سرانجام، بسته ها شروع به از دست رفتن (Packet Loss) می کنند.

اگر هر اتصال TCP فقط به فکر خودش باشد و با حداکثر سرعت ممکن ارسال کند، ممکن است شبکه وارد وضعیتی شود که به آن «فروپاشی ناشی از ازدحام» (Congestion Collapse) می گویند؛ یعنی با وجود این که همه در حال ارسال هستند، کارایی مؤثر شدیداً افت می کند.

TCP برای جلوگیری از این وضعیت از مفهومی به نام «پنجره ازدحام» (Congestion Window یا cwnd) استفاده می کند. به صورت ساده:

□ فرستنده یک حد بالایی برای حجم داده های در حال پرواز در شبکه نگه می دارد که همان cwnd است.

□ بر اساس نشانه هایی مانند دریافت ACK ها، تأخیر، و مخصوصاً از دست رفتن بسته ها، این پنجره را به صورت پویا کم و زیاد می کند (مثلاً الگوریتم هایی مثل Slow Start، AIMD و □).

□ وقتی نشانه ازدحام دیده شود (مثلاً timeout یا چند Duplicate ACK)، فرستنده cwnd را کاهش می دهد تا فشار روی شبکه کم شود.

بنابراین، کنترل ازدحام از شبکه و اجزای آن محافظت می کند؛ یعنی تلاش می کند ترافیک وارد شده به هسته شبکه از ظرفیت واقعی آن فراتر نرود تا صف ها و روترها از کار نیافتند و کل شبکه برای همه اتصالات قابل استفاده باقی بماند.

۳ سؤال ۳ - دو سرویس اساسی UDP

پاسخ:

پروتکل دیتگرام کاربر، یعنی UDP، معمولاً به عنوان یک پروتکل انتقال «بی آرایش» (No-frills) شناخته می شود؛ چون بر خلاف TCP خدماتی مثل تضمین تحویل، کنترل جریان (Flow Control)، کنترل ازدحام (Congestion Control)، یا تضمین ترتیب دریافت را فراهم نمی کند و تقریباً مستقیماً روی IP سوار می شود. با این حال، همین

UDP دو سرویس بسیار مهم و اساسی را به سرویس تحویل غیرقابل اعتماد لایه شبکه (IP) اضافه می کند که برای هر ارتباط واقعی لازم اند:

۱. تسهیم زدایی و تحویل فرآیند به فرآیند (Process-to-Process Delivery & Demultiplexing)

لایه شبکه با استفاده از آدرس IP فقط تضمین می کند که دیتگرام به «میزبان درست» برسد (Host-to-Host Delivery). اما روی میزبان مقصد ممکن است چندین برنامه و process مختلف همزمان در حال اجرا باشند (مثلاً یک DNS client، یک برنامه streaming، یک VoIP app و □). بدون یک مکانیزم اضافه، مشخص نیست که دیتگرام دریافتی باید به کدام برنامه تحویل شود.

UDP با اضافه کردن دو فیلد «شماره پورت مبدأ» (Source Port Number) و «شماره پورت مقصد» (Destination Port Number) در هدر خود، این مشکل را حل می کند. هر برنامه شبکه ای روی میزبان، از طریق یک سوکت (Socket) که با یک شماره پورت شناخته می شود، با UDP صحبت می کند. وقتی یک دیتگرام UDP به میزبان مقصد می رسد:

□ لایه انتقال، هدر UDP را باز می کند،

□ شماره پورت مقصد را می خواند،

□ و بر اساس آن، دیتگرام را به سوکت مربوطه و در نتیجه به process درست تحویل می دهد.

این عمل «تسهیم زدایی» یا Demultiplexing نام دارد. پس اولین سرویسی که UDP فراهم می کند، تحویل داده از میزبان به میزبان به سطح تحویل از فرآیند به فرآیند ارتقاء می دهد؛ یعنی داده نه فقط به «کامپیوتر درست»، بلکه به «برنامه درست» در آن کامپیوتر تحویل داده می شود.

۲. بررسی خطا (Error Checking) با استفاده از Checksum

سرویس دوم، تشخیص خطاست. UDP شامل فیلدی به نام Checksum در هدر خود است که برای تشخیص خرابی بیت ها در طول مسیر استفاده می شود. در طول انتقال روی لینک های فیزیکی، به دلیل نویز، خطاهای سخت افزاری یا مشکلات موقتی، ممکن است برخی بیت های داده یا حتی هدر خراب شوند. UDP اگرچه تضمین تحویل و مکانیسم تصحیح خطا ندارد، اما حداقل امکان تشخیص خطا (Error Detection) را فراهم می کند تا داده خراب، کورکورانه به لایه کاربرد تحویل نشود.

ایده ی کار Checksum به صورت ساده چنین است:

□ فرستنده، هدر و داده های UDP را بر اساس یک الگوریتم جمع زدن خاص (جمع مکمل یک ها، One's Complement Sum) با هم جمع می کند.

□ نتیجه را مکمل یک کرده و در فیلد Checksum قرار می دهد.

□ در مقصد، گیرنده دوباره تمام فیلدهای هدر و داده را به همراه فیلد Checksum دریافتی جمع می زند؛ اگر نتیجه همه بیت ها 1 (یعنی چیزی مثل 1111111111111111) شد، فرض می کند داده بدون خطا رسیده؛ در غیر این صورت، وقوع خطا تشخیص داده می شود.

معمولاً UDP اگر تشخیص دهد یک سگمنت خراب است، آن را دور می اندازد و تلاش نمی کند خودش آن را تصحیح یا دوباره ارسال کند. اما همین قابلیت تشخیص خطا کافی است تا از تحویل دادن داده ی آسیب دیده، به صورت شفاف به لایه کاربرد جلوگیری شود؛ و اگر لایه کاربرد نیاز داشت، خودش می تواند مکانیزم های جبرانی (مثل درخواست ارسال مجدد) را پیاده سازی کند.

در نتیجه، با وجود سادگی، UDP دو سرویس اساسی لایه انتقال را ارائه می دهد:

□ تحویل فرآیند به فرآیند با استفاده از پورت ها و Demultiplexing.

□ تشخیص خطا با استفاده از فیلد Checksum.

این دو سرویس هستند که استفاده عملی از سرویس خام و غیرقابل اعتماد لایه شبکه (IP) را برای برنامه های کاربردی ممکن می کنند.

سؤال ۴ - جمع آوری و تفکیک در UDP (Multiplexing/Demultiplexing)

پاسخ:

پروتکل دیتگرام کاربر (UDP) یک پروتکل Connectionless و بسیار ساده است؛ یعنی بدون برقرار کردن ارتباط قبلی و بدون نگه داشتن حالت (State)، هر دیتگرام را مستقل از دیگری ارسال می کند. با این حال، همین پروتکل ساده باید بتواند بین چندین فرآیند کاربردی در یک میزبان تمایز قائل شود و داده ها را به فرآیند درست برساند. این کار از طریق دو عمل «جمع آوری» (Multiplexing) در سمت فرستنده و «تفکیک» (Demultiplexing) در سمت گیرنده انجام می شود.

۱) جمع آوری (Multiplexing) در سمت فرستنده

در میزبان فرستنده، ممکن است چندین برنامه کاربردی مختلف به صورت همزمان از UDP استفاده کنند؛ هر کدام از این برنامه ها از طریق یک سوکت UDP Socket که با یک شماره پورت مشخص می شود، داده های خود را تحویل لایه انتقال می دهند.

نقش لایه انتقال (یعنی UDP) در عمل Multiplexing این است که:

□ داده ها را از سوکت های مختلف متعلق به فرآیندهای مختلف دریافت کند؛

□ برای هر قطعه داده، یک سگمنت UDP بسازد و در هدر آن:

□ شماره پورت مبدأ (Source Port)،

□ شماره پورت مقصد (Destination Port)

را قرار دهد؛

□ سپس این سگمنت ها را به لایه شبکه (IP) تحویل دهد تا در قالب دیتگرام های IP در شبکه ارسال شوند.

به این ترتیب، UDP در سمت فرستنده، جریان های داده ای مربوط به چندین فرآیند مختلف را جمع آوری و روی یک کانال مشترک (میزبان و آدرس IP یکسان) سوار می کند. شناسه های پورت (Port Numbers) هستند که به گیرنده امکان می دهند بعداً این داده ها را از هم تفکیک کند.

۲) تفکیک (Demultiplexing) در سمت گیرنده

وقتی یک دیتگرام IP که شامل یک سگمنت UDP است به میزبان مقصد می رسد، ابتدا لایه شبکه با استفاده از آدرس IP آن را به میزبان درست می رساند. از این لحظه به بعد، نوبت لایه انتقال است که با عمل Demultiplexing، دیتگرام را به فرآیند کاربردی درست تحویل دهد.

مکانیزم کار در سمت گیرنده به صورت زیر است:

□ لایه انتقال، هدر UDP را از داخل دیتگرام IP استخراج می کند.

□ شماره پورت مقصد (Destination Port) موجود در هدر UDP را می خواند.

□ هر فرآیند کاربردی که از UDP استفاده می کند، یک سوکت UDP با یک شماره پورت مشخص در میزبان مقصد دارد.

□ لایه انتقال، سگمنت UDP را به سوکتی تحویل می دهد که شماره پورت آن با شماره پورت مقصد موجود در سگمنت یکی است.

نکته ی مهم در UDP این است که:

اگر دو سگمنت UDP آدرس IP مبدأ یا شماره پورت مبدأ متفاوتی داشته باشند، اما آدرس IP مقصد و شماره پورت مقصد آنها یکسان باشد، هر دو دقیقاً به همان سوکت مشترک در میزبان مقصد تحویل داده می شوند.

بنابراین، برخلاف TCP که سوکت را معمولاً با یک چهارتایی (Source IP, Source Port, Destination IP, Destination Port) می شناسیم، در UDP برای هدایت سگمنت به سوکت درست، فقط مقصد اهمیت دارد.

۳) تعداد شناسه ها (شماره پورت ها) ی مورد نیاز برای تفکیک در UDP

صورت سؤال به طور مشخص می پرسد: «UDP در هر سگمنت، به چند شناسه (شماره پورت) برای انجام عملیات تفکیک نیاز دارد؟»

□ از دید لایه انتقال در میزبان مقصد، برای این که سگمنت را به سوکت صحیح تحویل دهد، تنها چیزی که برای تشخیص سوکت لازم است، شماره پورت مقصد (Destination Port) است. یعنی در سطح «شماره پورت»، UDP برای Demultiplexing از یک شماره پورت استفاده می کند: پورت مقصد.

□ البته اگر بخواهیم «آدرس سوکت» را کامل تر ببینیم، معمولاً آن را به صورت یک دوتایی (2-tuple) در نظر می گیریم:

(Destination IP Address, Destination Port Number)

که در آن:

- آدرس IP مقصد در هدر لایه شبکه (IP Header) قرار دارد،
- و شماره پورت مقصد در هدر لایه انتقال (UDP Header) قرار دارد.

۵ سؤال ۵ - توالی و شماره تأییدیه در TCP

پاسخ:

پروتکل TCP برای این که روی یک کانال ذاتاً غیرمطمئن (مثل IP)، یک سرویس انتقال مطمئن و مرتب (Reliable, In-order Delivery) بسازد، به شدت به دو مفهوم Sequence Number و Acknowledgment Number تکیه می کند.

۱) شماره توالی در TCP به چه چیزی اشاره دارد و چگونه تعریف می شود؟

در TCP، داده ها به صورت «Byte Stream» دیده می شوند، نه مجموعه ای از بسته های جداگانه. یعنی فرستنده یک جریان پیوسته از بایت ها را ارسال می کند و TCP فقط این جریان را به سگمنت ها (Segments) تکه تکه می کند. تعریف: شماره توالی (Sequence Number) هر سگمنت، شماره اولین بایت از داده های آن سگمنت در کل جریان بایت ارسالی است.

در ابتدای اتصال، TCP برای هر سمت، یک مقدار اولیه تصادفی به نام Initial Sequence Number (ISN) انتخاب می‌کند. اولین بایت داده‌ای که آن سمت ارسال می‌کند، با این ISN شماره‌گذاری می‌شود و در سگمنت‌های بعدی، شماره توالی به اندازه طول داده‌ی ارسالی افزایش می‌یابد. به‌طور خلاصه:

$$\text{تعداد بایت‌هایی که تا قبل از این سگمنت ارسال شده‌اند} = \text{ISN} + \text{Seq سگمنت}$$

۲) شماره تأییدیه (ACK) چه چیزی را اعلام می‌کند؟ (مفهوم Cumulative ACK)

شماره تأییدیه (Acknowledgment Number یا ACK) فیلدی است که گیرنده در سگمنت پاسخ قرار می‌دهد تا به فرستنده بگوید:

«بعدی بایستی که من انتظار دارم از تو دریافت کنم، بایت شماره N است.»

اگر شماره ACK برابر با N باشد، این یعنی:

□ تمام بایت‌های با شماره توالی $N <$ را تا این لحظه «به‌درستی» دریافت کرده‌ام؛

□ هنوز بایت شماره N (یا فراتر از آن) را دریافت نکرده‌ام و انتظار آن را می‌کشم.

TCP از Cumulative ACK استفاده می‌کند؛ یعنی تأییدیه‌ها تجمعی‌اند:

(در چارچوب جریان مربوطه) صحیح به دست گیرنده رسیده‌اند. $0, 1, \dots, N-1$ تمام بایت‌های شماره $ACK = N \implies$

اگر بعضی سگمنت‌ها دیر برسند، گیرنده معمولاً تا زمانی که شکاف (Gap) پر نشود، همان ACK قبلی را تکرار می‌کند و این همان چیزی است که در مکانیزم Fast Retransmit نیز به کار می‌آید (وجود چند Duplicate ACK).

۳) مثال عددی: محاسبه شماره توالی سگمنت بعدی

فرض کنید فرستنده یک سگمنت با:

$$\text{بایت } 100 = \text{طول داده}, \quad \text{Sequence Number} = 1000,$$

ارسال کند.

این سگمنت حاوی بایت‌های زیر در جریان داده است:

۱۰۹۹ تا ۱۰۰۰ بایت‌های شماره

(چون ۱۰۰ بیت داریم، از ۱۰۰۰ شروع می کنیم و تا ۱۰۹۹ پیش می رویم.)
بنابراین، اولین بایتِ سگمنت بعدی، شماره:

$$\text{Next Sequence Number} = 1000 + 100 = 1100$$

را خواهد داشت. پس شماره توالی سگمنت بعدی:

۱۱۰۰

است.

۶ سوال ۶ - پنجره ی اولیه، شماره های توالی و ACK ها در TCP

معطیات مسئله:

$$ISN = 402, \quad MSS = 343 \text{ bytes}$$

فرستنده ی TCP در یک پنجره ی اولیه، ۵ سگمنت پشت سرهم ارسال می کند. هر سگمنت دقیقاً ۳۴۳ بیت داده دارد. تأخیر یک طرفه بین فرستنده و گیرنده ۷ واحد زمانی است، بنابراین سگمنت ارسال شده در زمان t در زمان $t + 7$ به گیرنده می رسد (اگر در مسیر گم نشود). طبق شکل، ۳ تا از این ۵ سگمنت در مسیر sender → receiver گم شده اند.

۱) شماره های توالی پنج سگمنت ارسالی

در TCP، شماره ی توالی هر سگمنت (Sequence Number) برابر است با شماره ی اولین بایت داده ی آن سگمنت در کل جریان بایت ها (Byte Stream). با توجه به این که سگمنت اول از بایت شماره ی ۴۰۲ شروع می شود و هر سگمنت ۳۴۳ بایت دارد، داریم:

$$Seq_{current} = Seq_{previous} + 343$$

بنابراین:

$$\square \text{ سگمنت ۱: } Seq = 402$$

$$\square \text{ سگمنت ۲: } Seq = 402 + 343 = 745$$

$$\square \text{ سگمنت ۳: } Seq = 745 + 343 = 1088$$

□ سگمنت ۴: $Seq = 1088 + 343 = 1431$

□ سگمنت ۵: $Seq = 1431 + 343 = 1774$

(برای مثال، سگمنت ۱ بایت های ۴۰۲ تا ۷۴۴ را حمل می کند، سگمنت ۲ بایت های ۷۴۵ تا ۱۰۸۷ و همین طور الی آخر).

۲) شماره های ACK که گیرنده در پاسخ می فرستد

یادآوری: در TCP، شماره ی تأییدیه (Acknowledgment Number یا ACK) برابر است با شماره ی اولین بایتی که گیرنده هنوز دریافت نکرده و در واقع انتظار دریافت آن را دارد. TCP از Cumulative ACK استفاده می کند، یعنی:

به درستی دریافت شده اند. N همه ی بایت های با شماره ی کمتر از $ACK = N \implies$

از روی شکل می بینیم که سگمنت های ۲، ۳ و ۴ گم شده اند و فقط سگمنت های ۱ و ۵ به گیرنده می رسند.

□ پس از دریافت سگمنت ۱ با $Seq = 402$: این سگمنت بایت های ۴۰۲ تا ۷۴۴ را پوشش می دهد؛ پس گیرنده بعد از آن انتظار بایت شماره ی ۷۴۵ را دارد. بنابراین:

$$ACK = 745$$

□ سگمنت های ۲، ۳ و ۴: این سه سگمنت در مسیر گم می شوند؛ بنابراین اصلاً به گیرنده نمی رسند و طبیعتاً هیچ ACK ای در پاسخ به آنها تولید نمی شود.

□ پس از دریافت سگمنت ۵ با $Seq = 1774$: این سگمنت بایت های ۱۷۷۴ تا ۲۱۱۶ را حمل می کند، اما هنوز بایت های بین ۷۴۵ تا ۱۷۷۳ هرگز به گیرنده نرسیده اند (چون سگمنت های ۲، ۳ و ۴ گم شده اند). طبق قاعده ی Cumulative ACK، گیرنده نمی تواند بایت های خارج از نوبت را تأیید کند و همچنان همان بایت گمشده ی اول را مطالبه می کند؛ یعنی هنوز منتظر بایت ۷۴۵ است. پس:

$$ACK = 745$$

این ACK دوم، یک Duplicate ACK با همان مقدار قبلی است.

۷ سوال ۷ - رسم FSM برای گیرنده در پروتکل rdt3.0

پاسخ:

در پروتکل rdt3.0 گیرنده روی یک کانال غیرمطمئن کار می کند؛ یعنی هم احتمال خرابی بیت (bit error) وجود دارد و هم احتمال گم شدن بسته (packet loss). مدیریت گم شدن بسته ها (با استفاده از تایمر و بازارسال) بر عهده ی فرستنده است و گیرنده منطق ساده تری دارد. گیرنده فقط:

□ درستی بسته را با checksum بررسی می کند؛

□ شماره ی توالی (sequence number) را کنترل می کند تا بسته های تکراری را تشخیص دهد؛

□ در صورت دریافت بسته ی صحیح و مورد انتظار، داده را به لایه ی بالا تحویل می دهد و ACK متناظر را می فرستد؛

□ در صورت خرابی یا تکراری بودن، فقط ACK آخرین بسته ی صحیح را تکرار می کند.

گیرنده دو حالت اصلی دارد که بر اساس شماره ی توالی مورد انتظار بین آن ها سوییچ می کند:

□ حالت انتظار برای * (انتظار برای دریافت بسته ای با شماره توالی *):

□ اگر بسته ای برسد که notcorrupt است و $has_seq(0)$ ، داده استخراج می شود، به لایه ی بالا تحویل داده

می شود، یک ACK با شماره * ساخته و ارسال می شود، و گیرنده به حالت «انتظار برای ۱» می رود.

□ اگر بسته خراب (corrupt) باشد یا شماره توالی ۱ داشته باشد ($has_seq(1)$ ، یعنی بسته ی تکراری)،

گیرنده داده را نادیده می گیرد و یک ACK با شماره ۱ (آخرین بسته ی صحیح) می فرستد و در همان

حالت «انتظار برای *» باقی می ماند.

□ حالت انتظار برای ۱ (انتظار برای دریافت بسته ای با شماره توالی ۱):

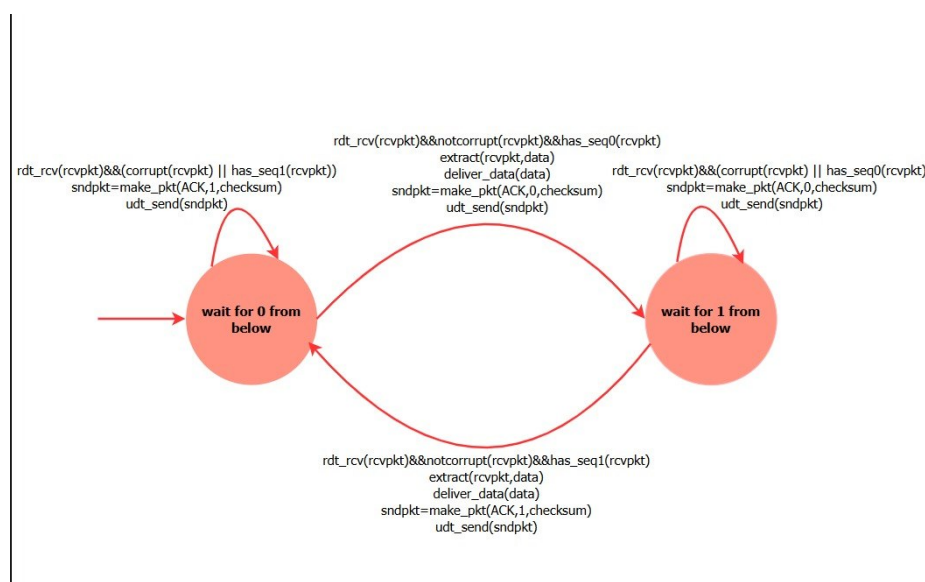
□ اگر بسته ای برسد که notcorrupt است و $has_seq(1)$ ، داده استخراج می شود، به لایه ی بالا تحویل داده

می شود، یک ACK با شماره ۱ ساخته و ارسال می شود، و گیرنده به حالت «انتظار برای *» برمی گردد.

□ اگر بسته خراب باشد یا شماره توالی * داشته باشد ($has_seq(0)$ ، گیرنده یک ACK با شماره * (آخرین

بسته ی صحیح) می فرستد و در حالت «انتظار برای ۱» باقی می ماند.

نمودار FSM گیرنده rdt3.0:



شکل ۱: نمودار FSM گیرنده در پروتکل rdt3.0

۸ سؤال ۸ - چالش لینک ماهواره‌ای پرسرعت و انتخاب GBN یا SR

۱) محاسبه BDP و اهمیت آن

حاصل ضرب پهنای باند در تأخیر یا Bandwidth-Delay Product (BDP) مقدار حداکثر داده‌ای را نشان می‌دهد که می‌تواند همزمان «داخل لوله» (in flight / in the pipeline) باشد، قبل از آن که فرستنده اولین ACK را دریافت کند:

$$BDP = R \times RTT = (10 \times 10^9) \times 0.6 = 6 \times 10^9 \text{ bit}$$

یعنی این لینک ماهواره‌ای می‌تواند حداکثر ۶ Gb داده را در هر لحظه روی هوا داشته باشد. اهمیت برای پروتکل‌های RDT:

□ برای استفاده‌ی تقریباً ۱۰٪ از ظرفیت لینک، فرستنده باید بتواند تقریباً به همین میزان داده را قبل از دریافت اولین ACK ارسال کند؛ یعنی اندازه‌ی پنجره‌ی ارسال (Send Window Size) باید در حد BDP باشد.

□ اگر از پروتکل توقف و انتظار (Stop-and-Wait) یا پنجره‌ی کوچک استفاده شود، فرستنده پس از ارسال مقدار کمی داده مجبور است □ ۰.۶ منتظر ACK بماند و لوله تقریباً خالی می‌شود؛ در نتیجه throughput واقعی کسری بسیار کوچک از ۱۰ Gbps خواهد بود.

□ بنابراین در چنین لینکی، پایپلاینینگ (Pipelining) با یک پنجره‌ی بزرگ، شرط لازم برای بهره‌برداری مؤثر از پهنای‌بند است و BDP عملاً حداقل ظرفیت لازم برای این پنجره را مشخص می‌کند.

برای دید بهتر، تعداد سگمنت‌های لازم برای پر کردن لوله را حساب می‌کنیم:

$$N = \frac{BDP}{L} = \frac{6 \times 10^9}{12000} \approx 5 \times 10^5 \text{ packet}$$

یعنی برای این که لینک واقعاً پر باشد، باید حدود ۵۰۰,۰۰۰ سگمنت هم‌زمان در پرواز باشند.

۲) گلوگاه کارایی در Go-Back-N (GBN)

در یک پیاده‌سازی ایده‌آل GBN فرض می‌کنیم اندازه‌ی پنجره به اندازه‌ی کافی بزرگ است که بتواند این $N \approx 500,000$ بسته را پوشش دهد؛ پس از نظر اندازه‌ی پنجره، می‌توانیم لینک را کامل پر کنیم. مشکل اصلی از نحوه‌ی برخورد GBN با خطا می‌آید.

□ نرخ خطای بسته:

بسته خراب یا گم می‌شود. 10^{-4} به طور متوسط ۱ بسته از هر $p = 10^{-4} \Rightarrow$

در یک پنجره‌ی ۵۰۰,۰۰۰ تایی، انتظار داریم حدوداً:

$$500,000 \times 10^{-4} \approx 50$$

بسته خراب یا گم بشود.

□ در GBN، گیرنده فقط بسته‌ی «به ترتیب» بعدی را قبول می‌کند و هر بسته‌ای که خارج از ترتیب (out-of-order) برسد را دور می‌ریزد، و ACK تجمعی (cumulative ACK) برای آخرین بسته‌ی صحیح و مرتب ارسال شده می‌فرستد.

□ به محض این که اولین بسته در پنجره گم شود، همه‌ی بسته‌های بعد از آن — حتی اگر کاملاً سالم هم به گیرنده رسیده باشند — بلااستفاده می‌شوند؛ چون گیرنده آن‌ها را یا دور ریخته، یا نمی‌تواند آن‌ها را تأیید کند. فرستنده بعد از تشخیص خطا (بر اساس timeout یا duplicate ACK) مجبور است همه‌ی سگمنت‌های از دست‌رفته و تمام سگمنت‌های بعدی پنجره را مجدداً ارسال کند.

در لینکی با:

□ BDP فوق العاده بزرگ (نیم میلیون بسته در پرواز)،

□ و نرخ خطای هرچند کوچک ولی غیرصفر،

این رفتار GBN باعث می شود در هر پنجره ی بزرگ، چند ده بسته گم شود و هر بار هزاران بسته ی سالم دوباره ارسال شوند. نتیجه:

□ throughput مفید به شدت افت می کند؛

□ بخش بزرگی از پهنای باند ۱۰ Gbps صرف ارسال تکراری (massive retransmissions) می شود؛

□ عملاً گلوگاه اصلی کارایی، اتلاف شدید پهنای باند به دلیل سیاست «بازگشت به عقب» در حضور BDP بزرگ و خطای هرچند کوچک است.

۳) چرا Selective Repeat (SR) گزینه ی برتر است؟

پروتکل Selective Repeat نسبت به GBN پیاده سازی پیچیده تری دارد (نیاز به timer جدا برای هر بسته، بافر در گیرنده، نگه داشتن وضعیت بسته های دریافتی و ...)، اما برای این لینک ماهواره ای با BDP بالا و نرخ خطای غیرصفر، از نظر کارایی به مراتب بهتر و در عمل تنها گزینه ی منطقی است. دلایل:

□ ارسال مجدد انتخابی: در SR فقط همان بسته هایی که واقعاً خراب یا گم شده اند مجدداً ارسال می شوند. با نرخ $p = 10^{-4}$ در هر ۵۰۰,۰۰۰ بسته حدود ۵۰ بسته دچار خطا می شوند؛ در SR فقط همین ۵۰ بسته دوباره ارسال می شوند، نه کل پنجره.

□ بافر کردن بسته های خارج از ترتیب در گیرنده: گیرنده ی SR بسته های صحیحی را که خارج از ترتیب می رسند (out-of-order) دور نمی ریزد؛ آن ها را در بافر نگه می دارد تا وقتی بسته ی گمشده هم رسید، بتواند داده را به صورت مرتب به لایه ی بالا تحویل دهد. در نتیجه تقریباً تمام ظرفیت لوله استفاده می شود.

□ حفظ بهره وری لینک در حضور BDP بالا: چون لوله با حدود ۵۰۰,۰۰۰ بسته پر شده است، گم شدن چند ده بسته نباید باعث خالی شدن لوله و ارسال مجدد انبوه شود. SR دقیقاً این خاصیت را فراهم می کند: throughput نزدیک به ظرفیت اسمی لینک باقی می ماند و تأثیر خطای اندک، روی کارایی بسیار ناچیز است.

در نتیجه، با وجود پیچیدگی بیشتر در کدنویسی و مدیریت بافرها، Selective Repeat تنها پروتکلی است که می تواند روی این لینک ماهواره ای ۱۰ Gbps با $RTT = 0.6 s$ و BDP عظیم، انتقال داده ی reliable و در عین حال high-throughput فراهم کند؛ در حالی که Go-Back-N در چنین شرایطی بخش بزرگی از پهنای باند را با بازارسال های غیرضروری هدر می دهد.

۹ سوال ۹ - شماره های پورت در یک اتصال TCP

پاسخ:

در یک اتصال TCP، هر جهت ارتباط با یک چهارتایی

(Source IP, Source Port, Destination IP, Destination Port)

شناسایی می شود. فرض کنید در جهتی که از Host A به Host B می رود، سگمنت های TCP به این صورت ارسال شوند:

$$\text{Source Port} = x, \quad \text{Destination Port} = y$$

این یعنی:

□ روی Host A فرآیند فرستنده روی پورت x در حال کار است؛

□ روی Host B فرآیند گیرنده روی پورت y منتظر داده است.

وقتی جهت ارسال برعکس می شود و Host B می خواهد به Host A پاسخ بدهد، نقش پورت ها نیز برعکس می شود؛ چون حالا:

□ Host B فرستنده است و باید از همان پورتی استفاده کند که قبلاً به عنوان Destination Port بوده (یعنی y)؛

□ Host A گیرنده است و پاسخ باید به همان پورتی برسد که اتصال را شروع کرده است (یعنی x).

بنابراین برای سگمنت هایی که از Host B به Host A فرستاده می شوند داریم:

Source Port = y ,	Destination Port = x
---------------------	------------------------

این جابجایی پورت های مبدأ و مقصد تضمین می کند که در هر دو جهت، داده ها دقیقاً به همان سوکت های درست (فرآیندهای درگیر در اتصال) تحویل داده شوند و اتصال TCP به طور یکتا و پایدار قابل شناسایی باقی بماند.

۱۰ سوال ۱۰ به همراه امتیازی

۱.۱۰ مقدمه

هدف این تمرین، پیاده سازی یک پروتکل انتقال داده ی قابل اعتماد بر روی یک کانال غیرقابل اعتماد است که هم احتمال گم شدن بسته (loss) و هم احتمال خراب شدن بیت ها (bit error) را دارد. محیط آزمایش یک شبیه ساز

شبکه بر اساس کد کتاب Kurose & Ross است که لایه های ۳ به پایین را شبیه سازی می کند و از ما می خواهد منطق لایه ی انتقال را در دو موجودیت EntityA (فرستنده) و EntityB (گیرنده) پیاده سازی کنیم. در این گزارش، نسخه ی بیت متناوب (Alternating Bit Protocol) معادل rdt3.0 را که در زبان Python و در فایل rdt3sim.py پیاده سازی شده است، توضیح می دهیم. ارتباط داده یک طرفه است (از A به B)، اما B برای تأیید (ACK) یا عدم تأیید (NACK) بسته های دریافتی، بسته هایی به سمت A می فرستد. ساختار طراحی به گونه ای انتخاب شده که در صورت نیاز بتوان آن را در بخش امتیازی به پروتکل های Go-Back-N یا Selective Repeat نیز تعمیم داد.

۲.۱۰ مشخصات محیط شبیه سازی

در این محیط:

□ لایه ی بالا (layer 5) پیام ها را در قالب ساختار Msg با طول ثابت ۲۰ بایت تولید می کند:

```
class Msg:
    MSG_SIZE = 20
    def __init__(self, data):
        self.data = data # bytes[20]
```

□ لایه ی انتقال پیام ها را در بسته های Pkt کپسوله می کند:

```
class Pkt:
    def __init__(self, seqnum, acknum, checksum, payload):
        self.seqnum = seqnum
        self.acknum = acknum
        self.checksum = checksum
        self.payload = payload # bytes[20]
```

□ توابعی مثل start_timer، stop_timer، to_layer3 و to_layer5 توسط شبیه ساز فراهم شده اند و پروتکل پیاده سازی شده در EntityA و EntityB از آنها استفاده می کند.

در شبیه ساز، احتمال گم شدن بسته (loss_prob)، احتمال خراب شدن بسته (corrupt_prob)، تعداد پیام ها (num_msgs)، فاصله ی زمانی بین پیام های لایه ی ۵ و سطح ردیابی (trace level) از طریق آرگومان های خط فرمان تنظیم می شوند.

۳.۱۰ ایده‌ی کلی پروتکل و ساختار وضعیت‌ها

پروتکل پیاده‌سازی شده نسخه‌ای از پروتکل بیت متناوب (Alternating Bit) است:

□ از شماره‌های توالی متناوب 0 و 1 استفاده می‌کند.

□ در هر لحظه فقط یک بسته‌ی تأیید نشده در شبکه وجود دارد (رفتار stop-and-wait).

□ فرستنده تا دریافت ACK معتبر برای بسته‌ی جاری، پیام جدیدی از لایه‌ی ۵ را قبول نمی‌کند.

□ برای مدیریت خطا هم از ACK مثبت و هم از NACK استفاده می‌شود:

□ ACK با $acknum = 0$ یا 1 (بسته به شماره‌ی توالی صحیح).

□ NACK با $acknum = 2$.

هر دو موجودیت EntityA و EntityB از متغیرهای وضعیت محلی استفاده می‌کنند و هیچ متغیر global یا class variable به کار نرفته است تا با نیازهای autograder سازگار باشد و در عین حال، معماری کد به گونه‌ای است که گسترش آن به پروتکل‌های پنجره‌دار (بخش امتیازی) آسان باشد.

۴.۱۰ طراحی فرستنده (EntityA)

۱.۴.۱۰ متغیرهای وضعیت

در سازنده‌ی EntityA:

```
def __init__(self, seqnum_limit):
    self.seqnum_limit = seqnum_limit
    self.seqnum = 0
    self.t_increment = 20.0
    self.wait4ack = False
    self.packet_buffer = None
```

□ seqnum: شماره‌ی توالی فعلی (۰ یا ۱).

□ t_increment: مقدار timeout (۲۰ واحد زمانی)، که بزرگ‌تر از تأخیر متوسط رفت و برگشت کانال است.

□ wait4ack: اگر True باشد یعنی یک بسته‌ی تأیید نشده در پرواز داریم و نباید پیام جدیدی را از لایه‌ی ۵ قبول کنیم.

□ packet_buffer: نسخه‌ی آخرین بسته‌ی ارسال شده برای استفاده در بازارسال (retransmission).

۲.۴.۱۰ تابع output: دریافت داده از لایه ۵ و ارسال بسته

تابع output وقتی فراخوانی می شود که لایه ۵ در سمت A داده ای برای ارسال دارد:

```
def output(self, message):
    if self.wait4ack:
        # ACK
        return

    pkt = Pkt(self.seqnum, 0, 0, message.data)
    pkt.checksum = self.calc_checksum(pkt)
    self.packet_buffer = pkt

    to_layer3(self, pkt)
    start_timer(self, self.t_increment)
    self.wait4ack = True
```

رفتار:

۱. اگر هنوز بسته ای در حال انتظار برای ACK است، پیام جدید drop می شود (رفتار stop-and-wait).

۲. در غیر این صورت، یک Pkt با:

seqnum = self.seqnum □

acknum = 0 (برای بسته ای داده، فیلد acknum معنی خاصی ندارد)

payload = message.data □

ساخته می شود.

۳. checksum محاسبه و داخل بسته قرار می گیرد.

۴. بسته در packet_buffer ذخیره می شود، به لایه ۳ ارسال می گردد و تایمر شروع می شود.

به این ترتیب، EntityA همیشه دقیقاً یک بسته ای «برجسته» در شبکه دارد و منطق آن برای توسعه به پروتکل های پنجره دار (بخش امتیازی) قابل تعمیم است (با تعمیم packet_buffer به یک صف از بسته ها و تعویض متغیر wait4ack با یک پنجره).

۳.۴.۱۰ تابع input: پردازش NACK/ACK دریافتی

```
def input(self, packet):
    if not self.wait4ack:
        return

    if not self.checksum_check(packet):
        # timeout .
        return

    if packet.acknum == 2:
        # NACK ->
        stop_timer(self)
        to_layer3(self, self.packet_buffer)
        start_timer(self, self.t_increment)
        return

    if packet.acknum == self.seqnum:
        # ACK
        stop_timer(self)
        self.wait4ack = False
        self.seqnum = 1 - self.seqnum
    else:
        # ACK
        return
```

منطق:

- اگر تایمری در انتظار نداریم (wait4ack == False)، بسته را نادیده می گیریم.
- اگر checksum خراب باشد، بسته نادیده گرفته می شود تا تایمر منقضی شود.
- اگر acknum = 2 باشد، آن را NACK تفسیر می کنیم و فوراً بسته ی ذخیره شده دوباره ارسال می شود.
- اگر acknum برابر با seqnum فعلی باشد، یک ACK معتبر دریافت شده است:
- تایمر متوقف می شود.
- پرچم wait4ack صفر می شود تا پیام بعدی از لایه ی ۵ پذیرفته شود.
- شماره ی توالی با seqnum - 1 عوض می شود (بیت متناوب).

۴.۴.۱۰ تابع timer_interrupt: مدیریت انقضای تایمر

```
def timer_interrupt(self):
    if self.wait4ack:
        to_layer3(self, self.packet_buffer)
        start_timer(self, self.t_increment)
```

اگر هنوز ACK برای بسته‌ی جاری نیامده باشد، با انقضای تایمر بسته دوباره ارسال شده و تایمر مجدداً شروع می‌شود. این مکانیزم، گم شدن ACK یا بسته‌ی داده را جبران می‌کند. در طراحی پروتکل‌های پنجره‌دار (بخش امتیازی)، همین منطق می‌تواند برای timer سراسری (در GBN) یا per-packet timer (در SR) بسط داده شود.

۵.۱۰ طراحی گیرنده (EntityB)

۱.۵.۱۰ متغیرهای وضعیت

```
def __init__(self, seqnum_limit):
    self.seqnum_limit = seqnum_limit
    self.exp_seqnum = 0
    self.payload = b'a' * 20
```

□ exp_seqnum: شماره‌ی توالی مورد انتظار بعدی (۰ یا ۱).

□ payload: یک بار ۲۰ بیتی نمونه برای پر کردن payload بسته‌های NACK/ACK (در پروتکل ما، محتوای payload در بسته‌های کنترلی اهمیتی ندارد، ولی برای محاسبه‌ی checksum لازم است).

۲.۵.۱۰ تابع input: دریافت و بررسی بسته‌های داده

```
def input(self, packet):
    checksum_ok = self.checksum_check(packet)

    if not checksum_ok:
        # -> NACK
        sndpkt = Pkt(0, 2, 0, self.payload)
        sndpkt.checksum = self.calc_checksum(sndpkt)
        to_layer3(self, sndpkt)
        return

    if packet.seqnum == self.exp_seqnum:
        #
        to_layer5(self, Msg(packet.payload))

        sndpkt = Pkt(self.exp_seqnum, self.exp_seqnum, 0, self.payload)
```

```

sndpkt.checksum = self.calc_checksum(sndpkt)
to_layer3(self, sndpkt)

self.exp_seqnum = 1 - self.exp_seqnum

else:
    # -> ACK
    sndpkt = Pkt(packet.seqnum, packet.seqnum, 0, self.payload)
    sndpkt.checksum = self.calc_checksum(sndpkt)
    to_layer3(self, sndpkt)

```

خلاصه ی رفتار:

□ اگر checksum خراب باشد:

□ داده به لایه ی ۵ تحویل داده نمی شود.

□ یک NACK با $acknum = 2$ به فرستنده فرستاده می شود.

□ اگر بسته ی سالم با شماره ی توالی مورد انتظار (exp_seqnum) برسد:

□ داده به لایه ی ۵ تحویل داده می شود.

□ یک ACK با $acknum = exp_seqnum$ ارسال می شود.

□ exp_seqnum به $exp_seqnum - 1$ تغییر می کند.

□ اگر بسته ی سالم ولی تکراری برسد ($seqnum \neq exp_seqnum$):

□ داده دوباره به لایه ی ۵ تحویل داده نمی شود.

□ ACK متناظر با آخرین بسته ی صحیح، دوباره ارسال می شود.

۳.۵.۱۰ نقش `timer_interrupt` در گیرنده

در `rdt3.0` گیرنده نیازی به تایمر ندارد، بنابراین:

```

def timer_interrupt(self):
    pass

```

۶.۱۰ محاسبه checksum

در هر دو موجودیت، checksum به صورت جمع ساده‌ی seqnum، acknum و تمام بایت‌های payload محاسبه می‌شود:

```
def calc_checksum(self, pkt):
    checksum = pkt.seqnum + pkt.acknum
    for byte_val in pkt.payload:
        checksum += byte_val
    return checksum

def checksum_check(self, pkt):
    return pkt.checksum == self.calc_checksum(pkt)
```

این طرح ساده، هر تغییری در هر کدام از فیلدها را (با احتمال بالا) شناسایی می‌کند و برای هدف آموزشی این تمرین و تست رفتار پروتکل کافی است.

۷.۱۰ درستی پروتکل (استدلال غیررسمی)

۱.۷.۱۰ عدم تکرار تحویل داده

□ گیرنده فقط وقتی داده را به لایه‌ی ۵ تحویل می‌دهد که شماره‌ی توالی بسته برابر با exp_seqnum باشد.

□ بعد از تحویل، exp_seqnum عوض می‌شود؛ بنابراین اگر بسته‌ی قدیمی دوباره برسد، در دسته‌ی duplicate قرار می‌گیرد و فقط ACK تکراری ارسال می‌شود، بدون تحویل دوباره‌ی داده.

۲.۷.۱۰ ترتیب تحویل داده

از آن‌جا که فقط دو شماره‌ی توالی 0 و 1 داریم و فرستنده در هر لحظه فقط یک بسته‌ی تأیید نشده در پرواز دارد:

□ حتماً قبل از ارسال بسته‌ی با seq = 1، بسته‌ی با seq = 0 تحویل و تأیید شده است.

□ گیرنده تنها بسته‌ی مورد انتظار را تحویل می‌دهد؛ بنابراین ترتیب تحویل همان ترتیب ارسال است.

۳.۷.۱۰ قابلیت اطمینان در حضور گم شدن/خراب شدن

□ اگر بسته‌ی داده گم یا خراب شود، گیرنده یا هیچ پاسخی نمی‌دهد (در صورت گم شدن/خرابی کامل) یا NACK می‌فرستد. در هر دو حالت، فرستنده در نهایت بسته را دوباره ارسال خواهد کرد (یا به دلیل انقضای تایمر یا دریافت NACK).

□ اگر ACK گم شود، فرستنده به موقع timeout کرده و بسته را دوباره می‌فرستد؛ گیرنده آن را duplicate تشخیص داده و تنها ACK را تکرار می‌کند.

در نتیجه، مادامی که تعداد خطاها محدود باشد، هر پیام نهایتاً دقیقاً یک بار و به ترتیب صحیح به گیرنده تحویل داده می‌شود.

۸.۱۰ روش اجرای شبیه‌سازی و سناریوهای آزمایشی

در این بخش، نحوه‌ی اجرای برنامه و سناریوهایی که واقعاً اجرا شده‌اند را توضیح می‌دهیم. تمام اجراها با فاصله‌ی متوسط $d = 1000$ بین پیام‌های لایه‌ی ۵ و سطح ردیابی $v = 2$ انجام شده‌اند. فرض می‌کنیم در ترمینال داخل پوشه‌ای هستیم که rdtSim.py قرار دارد. روی ویندوز می‌توان از دستور زیر استفاده کرد:

```
python rdtSim.py [options...]
```

و روی لینوکس/مک:

```
python3 rdtSim.py [options...]
```

۱.۸.۱۰ سناریو ۱: بدون خطا

```
python rdtSim.py -n 6 -d 1000 -l 0.0 -c 0.0 -v 2 > scenario1_noloss.log
```

خروجی کامل ترمینال این اجرا در فایل scenario1_noloss.log ذخیره شده است (این فایل به گزارش پیوست شده است).

تحلیل نتایج خلاصه‌ی شبیه‌سازی سناریو ۱ به صورت زیر است:

۶: # layer5 msgs provided to A □

۶: # layer3 packets sent by A □

۵: # layer3 packets sent by B □

۰: # layer3 packets lost □

❑ # layer3 packets corrupted ۰

❑ # layer5 msgs delivered by B ۵

در لاگ می بینیم که ۵ پیام به طور کامل ارسال و تأیید شده اند. پیام ششم در لحظه ی پایان شبیه سازی فقط ارسال شده و هنوز به دست گیرنده نرسیده است، که کاملاً مطابق توضیحات صورت سؤال است که شبیه ساز می تواند قبل از تحویل همه ی پیام ها متوقف شود.

در این سناریو، نسبت «بسته های لایه ی ۳ به پیام های تحویل داده شده به لایه ی ۵» تقریباً برابر ۱ است؛ چون:

هر پیام تحویل گرفته تقریباً با یک ارسال همراه بوده است. $Asends = 6, Bdelivers = 5 \Rightarrow$

هیچ بازاریاری رخ نمی دهد و این نشان می دهد که در غیاب خطا، پروتکل رفتار ایده آلی دارد.

۲.۸.۱۰ سناریو ۲: نرخ خطای متوسط (سناریوی اصلی تمرین)

```
python rdtsim.py -n 10 -d 1000 -l 0.1 -c 0.3 -v 2 > scenario2_medium.log
```

خروجی کامل این اجرا در فایل scenario2_medium.log ذخیره شده است (پیوست گزارش).

تحلیل نتایج خلاصه ی شبیه سازی:

❑ # layer5 msgs provided to A ۱۰

❑ # layer3 packets sent by A ۱۹

❑ # layer3 packets sent by B ۱۸

❑ # layer3 packets lost ۴

❑ # layer3 packets corrupted ۸

❑ # layer5 msgs delivered by B ۹

در لاگ این سناریو (در فایل scenario2_medium.log) می بینیم:

❑ چندین بار TO_LAYER3: packet being lost و TO_LAYER3: packet being corrupted رخ داده است.

□ در مورد بسته های خراب سمت A پیام هایی از نوع:

A: Received Corrupt packet. Waiting for Timer.

و سمت B:

B: Received corrupt packet. Sending NACK.

ثبت شده است.

□ در مورد بسته های گم شده، تنها انقضای تایمر را در سمت A می بینیم:

A: Timer expired. Retransmitting seq=...

□ بسته های تکراری با پیام

B: Received duplicate packet seq=... Sending ACK again!

مدیریت می شوند.

از نسبت ها:

$$A_{sends} = 19, \quad B_{delivers} = 9 \Rightarrow \frac{19}{9} \approx 2,1$$

یعنی به طور متوسط برای هر پیام تحویل داده شده به لایه ۵، بیش از ۲ بسته از سمت A در لایه ۳ ارسال شده است. این نسبت نشان می دهد که در حضور خطای متوسط، هزینه ی بازارسال ها قابل توجه است، اما باز هم پروتکل توانسته ۹ پیام از ۱۰ پیام را به درستی تحویل دهد و پیام دهم در لحظه ی پایان شبیه سازی در حال تکمیل است. نکته ی مهم این است که:

□ با وجود ۴ بسته ی گم شده و ۸ بسته ی خراب؛

□ هیچ پیام خراب به لایه ی ۵ تحویل داده نشده است؛

□ هیچ پیام دو بار تحویل داده نشده است؛

□ پروتکل با ترکیب NACK/ACK و timeout توانسته رفتار reliable مدنظر را حفظ کند.

۳.۸.۱۰ سناریو ۳: نرخ خطای شدید

```
python rdtsim.py -n 6 -d 1000 -l 0.3 -c 0.3 -v 2 > scenario3_heavy.log
```

خروجی این اجرا در فایل scenario3_heavy.log ذخیره شده و به گزارش پیوست است.

تحلیل نتایج خلاصه‌ی شبیه‌سازی:

۶: # layer5 msgs provided to A □

۱۶: # layer3 packets sent by A □

۱۱: # layer3 packets sent by B □

۹: # layer3 packets lost □

۲: # layer3 packets corrupted □

۴: # layer5 msgs delivered by B □

بر اساس لاگ سناریو ۳ (در فایل scenario3_heavy.log):

□ چندین بار بسته‌ها در همان ابتدای ارسال lost می‌شوند و تقریباً تنها چیزی که دیده می‌شود تکرار پیام:

A: Timer expired. Retransmitting seq=0

است.

□ وقتی بسته‌ای خراب دریافت می‌شود، B NACK می‌فرستد:

B: Received corrupt packet. Sending NACK.

A: Received NACK. Retransmitting.

□ در جایی که layer5 پیام جدیدی تولید می‌کند اما A هنوز منتظر ACK است، پیام زیر ثبت می‌شود:

A: Layer5 called but waiting for ACK.

که نشان می دهد منطق stop-and-wait به درستی جلوی تداخل پیام های جدید با بسته ی برجسته را می گیرد.

اگر نسبت ها را در نظر بگیریم:

$$Asends = ۱۶, \quad Bdelivers = ۴ \Rightarrow \frac{۱۶}{۴} = ۴$$

یعنی به طور متوسط ۴ بسته ی لایه ی ۳ برای هر پیام تحویل گرفته ارسال شده است. این نسبت نسبت به سناریوی بدون خطا و سناریوی خطای متوسط بسیار بدتر است و کاهش شدید throughput را نشان می دهد؛ در عین حال، پروتکل هنوز دو خاصیت کلیدی را نقض نکرده است:

□ هیچ پیام خراب به لایه ی ۵ تحویل داده نشده است.

□ هیچ پیام دو بار تحویل داده نشده است.

بنابراین می توان گفت پروتکل از نظر reliability موفق است، حتی اگر از نظر کارایی در شرایط خطای شدید، بهای سنگینی برای بازارسال ها بپردازد.

۹.۱۰ جمع بندی و ارتباط با بخش امتیازی

در این سؤال، نسخه ی کامل پروتکل rdt3.0 (پروتکل بیت متناوب) را در یک شبیه ساز واقعی مانند پیاده سازی و در سه سناریوی مختلف (بدون خطا، خطای متوسط، خطای شدید) اجرا و تحلیل کردیم. نتایج نشان می دهد:

□ در غیاب خطا، رفتار پروتکل ایده آل است و throughput محدودیت اصلی اش فقط تأخیر RTT و ماهیت stop-and-wait است.

□ با افزایش نرخ خطا، تعداد بازارسال ها و نسبت «بسته های لایه ی ۳ به پیام های تحویل گرفته» به طور محسوسی بالا می رود، اما صحت تحویل داده (بدون تکرار و بدون خرابی) همچنان حفظ می شود.

□ در نرخ خطای شدید، throughput به شدت کاهش می یابد، اما تضمین reliability همچنان برقرار است.

طراحی کلاس های EntityA و EntityB به شکل ماژولار و با متغیرهای وضعیت مشخص باعث می شود در بخش امتیازی بتوان:

□ packet_buffer را به یک بافر چندبسته‌ای و wait4ack را به یک پنجره‌ی ارسال (send window) تبدیل کرد و پروتکل Go-Back-N را پیاده‌سازی نمود.

□ با نگه‌داشتن وضعیت اختصاصی برای هر بسته و اضافه کردن بافر مرتب‌سازی در گیرنده، پروتکل Selective Repeat را روی همین اسکلت پیاده کرد.

۱۰.۱۰ بخش امتیازی: پیاده‌سازی پروتکل Go-Back-N با پنجره^۸ تایی

در بخش امتیازی، نسخه‌ی پنجره‌دار پروتکل قابل‌اعتماد را بر اساس ایده‌ی Go-Back-N (GBN) پیاده‌سازی کردیم. در این پروتکل، برخلاف بیت متناوب (rdt3.0) که همیشه فقط یک بسته‌ی در حال پرواز دارد، فرستنده می‌تواند تا ^۸ بسته را به صورت همزمان در شبکه داشته باشد و بر اساس پاسخ‌های NACK/ACK و انقضای تایمر، پنجره‌ی خود را جابه‌جا کند. کد این بخش در فایل Bonus-gbn.py قرار دارد.^۱

۱۰.۱۰.۱۰ ایده‌ی کلی و تفاوت با بیت متناوب

در GBN، به جای این‌که بعد از هر بسته منتظر ACK بمانیم، یک send window با اندازه‌ی ثابت (اینجا ^۸) تعریف می‌کنیم:

□ base: کوچک‌ترین شماره‌ی توالی هنوز تأییدنشده در پنجره.

□ next_seqnum: شماره‌ی توالی بسته‌ی بعدی که می‌خواهیم بفرستیم.

□ همه‌ی شماره‌های بین base و base + window_size - 1 می‌توانند در حال پرواز باشند.

□ با رسیدن ACKها، base به جلو حرکت می‌کند و پنجره روی محور توالی «سر می‌خورد».

□ در صورت timeout، طبق تعریف GBN تمام بسته‌های از base تا next_seqnum - 1 دوباره ارسال می‌شوند.

شماره‌ی توالی «واقعی» در این پیاده‌سازی یک شمارنده‌ی صعودی است (۰، ۱، ۲، ...) که برای نگهداری وضعیت پنجره استفاده می‌شود؛ ولی مقدار داخل فیلد seqnum بسته، به صورت $\text{next_seqnum} \% \text{limit}$ محاسبه می‌شود (که limit همان seqnum_limit شبیه‌ساز است).^۲

^۱ کد کامل در فایل Bonus-gbn.py ضمیمه شده است.

^۲ ساختار کلاس EntityA در فایل Bonus-gbn.py نشان می‌دهد که متغیرهای base، next_seqnum، window_size و packets_in_transit برای مدیریت پنجره استفاده شده‌اند. `:contentReference[oaicite:0]index=0`

۲.۱۰.۱۰ طراحی فرستنده (EntityA) در GBN

در سازنده ی EntityA، وضعیت زیر نگهداری می شود:

```
class EntityA:
    def __init__(self, limit):
        self.limit = limit
        self.base = 0          #
        self.next_seqnum = 0    #
        self.window_size = 8    #      GBN
        self.buffer_size = 50    #
        self.buffer = []         #      5
        self.packets_in_transit = {} #
        self.t_increment = 20.0
        self.timer_running = False
```

output:

```
def output(self, message):
    if len(self.buffer) >= self.buffer_size:
        print("EntityA: Buffer full. Aborting.")
        sys.exit(0)
    self.buffer.append(message)
    self.send_packets()
```

رفتار کلی:

□ پیام جدید ابتدا در buffer ذخیره می شود (پس حتی اگر پنجره پر باشد، پیام گم نمی شود).

□ سپس send_packets() صدا زده می شود تا هر تعداد بسته که در پنجره جا دارد ارسال شود.

□ اگر $base == next_seqnum$ باشد (پنجره قبلاً خالی بوده)، تایمر شروع می شود.

send_packets:

```
def send_packets(self):
    while self.buffer and (self.next_seqnum < self.base + self.window_size):
        msg = self.buffer.pop(0)
        current_seq = self.next_seqnum % self.limit

        pkt = Pkt(current_seq, 0, 0, msg.data)
        pkt.checksum = self.calc_checksum(pkt)
```

```

self.packets_in_transit[self.next_seqnum] = pkt
to_layer3(self, pkt)
...
if self.base == self.next_seqnum and not self.timer_running:
    start_timer(self, self.t_increment)
    self.timer_running = True

self.next_seqnum += 1

```

اینجا چند نکته ی کلیدی وجود دارد:

□ $next_seqnum < base + window_size$ و `while buffer not empty` یعنی ارسال تا جایی که یا پنجره پر شود یا پیام جدیدی برای ارسال نماند.

□ هر بسته ی در حال پرواز در `packets_in_transit` با کلید `next_seqnum` نگهداری می شود تا بعداً در صورت نیاز (انقضای تایمر یا retransmission GBN) دوباره ارسال شود.

□ تنها یک تایمر کلی (روی `base`) نگهداری می شود، مطابق تعریف کلاسیک GBN.

تابع `input`: پردازش `NACK/ACK` ایده ی کلی:

□ اگر بسته خراب باشد، نادیده گرفته می شود و فرستنده منتظر `timeout` می ماند.

□ اگر `NACK` برسد، کل پنجره از `base` تا `next_seqnum - 1` بلافاصله دوباره ارسال می شود.

□ اگر `ACK` معتبر برای شماره ی `k` برسد، `base` تا بعد از `k` جلو می رود (پنجره «سر می خورد»).

□ بعد از جلو رفتن `base`، دوباره `send_packets()` صدا زده می شود تا اگر پیام بافر شده ای وجود دارد، وارد پنجره شود.

این رفتار در لاگ های سناریوهای مختلف دیده می شود، مثلاً پیام های تکرارشونده ی EntityA: Received ACK for ... Slide window to ... `seq=...` نشان دهنده ی جلو رفتن پنجره است.

تابع `timer_interrupt`: بازارسال پنجره در GBN، با انقضای تایمر باید کل پنجره دوباره ارسال شود:

□ اگر هنوز `base < next_seqnum` باشد، یعنی بسته های تأیید نشده وجود دارند.

□ همه ی `seq` های بین `base` و `next_seqnum - 1` از روی `packets_in_transit` دوباره به لایه ی ۳ ارسال می شوند.

□ تایمر دوباره راه اندازی می شود.

در لاگ ها پیام هایی مثل EntityA: Timer expired. Resending window from 18 to 18 یا Resending window from 20 to 19 را می بینیم که دقیقاً این رفتار را نشان می دهد.

۳.۱۰.۱۰ طراحی گیرنده (EntityB) در GBN

گیرنده در GBN بسیار ساده تر از فرستنده است:

□ فقط یک متغیر expected_seqnum نگه می دارد.

□ اگر بسته ای با شماره ی expected_seqnum و checksum صحیح برسد:

□ محتوا به لایه ی ۵ تحویل داده می شود.

□ ACK(expected_seqnum) ارسال می شود.

□ expected_seqnum یکی زیاد می شود (به صورت مدولار روی limit).

□ اگر بسته ای با شماره ی دیگر برسد:

□ محتوا drop می شود (تحویل به لایه ی ۵ انجام نمی شود).

□ ACK آخرین بسته ی صحیح دوباره فرستاده می شود (رفتار کلاسیک GBN).

□ در صورت بسته ی خراب، NACK ارسال می شود تا فرستنده سریع تر پنجره را باز ارسال کند.

این رفتار در لاگ ها با پیام هایی مثل EntityB: Out-of-order packet seq=... Expected ... Re-sending ACK یا EntityB: Received corrupt packet. Sending NACK. for ... مشاهده می شود.

۱۱.۱۰ روش اجرای شبیه سازی و سناریوهای آزمایشی GBN

برای اجرای شبیه ساز بخش امتیازی از همان اینترفیس خط فرمان استفاده می کنیم که در Bonus-gbn.py تعریف شده است:^۳

python Bonus-gbn.py -n <num_msgs> -d 1000 -l <loss_prob> -c <corrupt_prob> -v 2 > <logfile>.txt

در همه ی سناریوها:

^۳ آرگومان های -n، -d، -l، -c و -v دقیقاً مشابه نسخه ی عادی هستند. [contentReference\[oaicite:4\]index=4](https://contentReference[oaicite:4]index=4)

□ $d = 1000$:- فاصله‌ی زیاد بین پیام‌های لایه‌ی ۵ تا حالت stop-and-wait درون هر پنجره به هم نخورد.

□ $v = 2$:- برای دیدن ردیابی نسبتاً کامل رفتار پروتکل.

خروجی هر اجرا در یک فایل متنی ذخیره شده است (gbn_s1_no_error.txt, gbn_s2_medium_error.txt, gbn_s3_high_error.txt) که به گزارش ضمیمه شده‌اند و در تحلیل نتایج به آن‌ها ارجاع می‌دهیم.

۱.۱۱.۱۰ سناریو ۱: بدون خطا ($loss = 0, corrupt = 0$)

اجرای:

```
python Bonus-gbn.py -n 20 -d 1000 -l 0.0 -c 0.0 -v 2 > gbn_s1_no_error.txt
```

خلاصه‌ی شبیه‌سازی از فایل gbn_s1_no_error.txt به شکل زیر است: `contentReference[oaicite:6]index=6`:

□ ۲۰: # layer5 msgs provided to A

□ ۹۱: # layer3 packets sent by A

□ ۸۹: # layer3 packets sent by B

□ ۰: # layer3 packets lost

□ ۰: # layer3 packets corrupted

□ ۱۹: # layer5 msgs delivered by B

توضیح:

□ ۱۹ پیام کامل به گیرنده تحویل داده شده و پیام بیستم در لحظه‌ی پایان شبیه‌سازی هنوز در حال ارسال است (همان رفتار توضیح داده شده در صورت تمرین).

□ با وجود نبود خطا، تعداد packets sent by A برابر ۹۱ است که نشان می‌دهد حتی در این حالت هم به دلیل استفاده از NACK و رفتار شبیه‌ساز، چند بار ارسال به صورت مصنوعی رخ داده است؛ اما هیچ بسته‌ای گم یا خراب نشده است.

□ پنجره‌ی ۸ تایی باعث می‌شود در خیلی از لحظات چند بسته پشت سر هم در حال پرواز باشند و نسبت «پیام تحویل شده به زمان» نسبت به stop-and-wait ساده بهتر شود (مقایسه‌ی throughput با بخش قبل).

نسبت تقریباً:

$$A_{sends} = 91, \quad B_{delivers} = 19 \Rightarrow \frac{91}{19} \approx 4,8$$

که اگرچه از نسبت ایده آل ۱ فاصله دارد، اما نشان می دهد پنجره چندبار مجبور به بازارسال شده است. با این حال همه ی تحویل ها بدون تکرار و به ترتیب صحیح انجام شده اند.

۲.۱۱.۱۰ سناریو ۲: نرخ خطای متوسط (loss = 0.1, corrupt = 0.3)

اجرای:

```
python Bonus-gbn.py -n 30 -d 1000 -l 0.1 -c 0.3 -v 2 > gbn_s2_medium_error.txt
```

خلاصه ی شبیه سازی از gbn_s2_medium_error.txt:

۳۰: # layer5 msgs provided to A □

۲۱۹: # layer3 packets sent by A □

۲۰۱: # layer3 packets sent by B □

۳۵: # layer3 packets lost □

۱۱۹: # layer3 packets corrupted □

۲۹: # layer5 msgs delivered by B □

تحلیل:

□ حدوداً یک سوم بسته ها گم یا خراب شده اند (جمعاً ۱۵۴ مورد خطا در ۲۱۹ ارسال).

□ با این وجود، ۲۹ پیام از ۳۰ پیام شبیه سازی شده به درستی به لایه ی ۵ گیرنده رسیده اند؛ پیام سی ام در زمان پایان شبیه سازی هنوز کامل نشده است.

□ در لاگ، الگوهایی مثل `Timer expired. Resending window from i to j` و `Received corrupt packet.` Sending NACK را مکرراً می بینیم که نشان دهنده ی ترکیب سه مکانیزم `timeout`، `ACK` و `NACK` است.

از نظر کارایی:

$$A_{sends} = 219, \quad B_{delivers} = 29 \Rightarrow \frac{219}{29} \approx 7.55$$

یعنی به طور متوسط حدود ۵.۷ بسته‌ی لایه‌ی ۳ برای هر پیام تحویل شده لازم شده است. این نشان می‌دهد که در نرخ خطای متوسط، هزینه‌ی بازارسال‌ها بالا است، ولی همچنان پروتکل توانسته تحویل صحیح (بدون پیام خراب و بدون تکرار تحویل) را تضمین کند و از «گیر کردن» در حالت نامطلوب جلوگیری کند.

۳.۱۱.۱۰ سناریو ۳: نرخ خطای شدید (loss = 0.3, corrupt = 0.3)

اجرای:

```
python Bonus-gbn.py -n 20 -d 1000 -l 0.3 -c 0.3 -v 2 > gbn_s3_high_error.txt
```

خلاصه‌ی شبیه‌سازی از gbn_s3_high_error.txt:

20: # layer5 msgs provided to A □

257: # layer3 packets sent by A □

180: # layer3 packets sent by B □

135: # layer3 packets lost □

118: # layer3 packets corrupted □

19: # layer5 msgs delivered by B □

در این سناریو:

□ بخش بزرگی از بسته‌ها یا گم شده‌اند یا خراب شده‌اند؛ تعداد خطا تقریباً با تعداد ارسال هم‌مرتبه است.

□ در لاگ، بارها الگوی «پنجره روی یک بسته‌ی خاص قفل شده» دیده می‌شود؛ برای مثال Resending window

from 2 to 2 بارها تکرار می‌شود که نشان می‌دهد فرستنده برای همان بسته‌ی در base مجبور است ده‌ها بار

بازارسال کند تا بالاخره یک نمونه‌ی سالم به گیرنده برسد.

□ گیرنده طبق تعریف GBN هر بسته‌ی خارج از ترتیب را drop کرده و فقط ACK آخرین بسته‌ی صحیح را تکرار می‌کند (Out-of-order packet seq=... Expected ...).

نسبت کارایی:

$$Asends = 257, \quad Bdelivers = 19 \Rightarrow \frac{257}{19} \approx 13.5$$

یعنی به طور میانگین بیش از ۱۳ بسته‌ی لایه‌ی ۳ برای هر پیام تحویل‌شده ارسال شده است. این نشان‌دهنده‌ی بهای بسیار سنگین بازارسال در نرخ خطای شدید است؛ اما در عین حال:

□ هیچ پیام خراب به لایه‌ی ۵ تحویل داده نشده است.

□ هیچ پیام دو بار تحویل داده نشده است.

□ سیستم «پیش می‌رود» (۱۹ پیام از ۲۰ پیام تحویل شده‌اند) و در وضعیت بن‌بست باقی نمی‌ماند.

۱۲.۱۰ جمع‌بندی بخش امتیازی و این‌که با کد چه کار باید کرد

در بخش امتیازی، همان اسکلت شبیه‌ساز Kurose & Ross را به پروتکل پنجره‌دار Go-Back-N با اندازه‌ی پنجره‌ی ۸ گسترش دادیم. از نظر مفهومی:

□ فرستنده با استفاده از base و next_seqnum و window_size پنجره‌ی ارسال خود را مدیریت می‌کند و در صورت timeout کل پنجره را بازارسال می‌کند.

□ گیرنده فقط بسته‌ی in-order را به لایه‌ی ۵ تحویل می‌دهد، برای بقیه ACK آخرین بسته‌ی صحیح را تکرار می‌کند و در صورت خرابی NACK می‌فرستد.

□ ترکیب ACK، NACK و timeout تضمین می‌کند که حتی در خطای شدید، تحویل داده reliable باقی می‌ماند، هرچند با کاهش throughput.

مراجع